

LLMOrchestrator

LLMOrchestrator

Un plano de control para cada agente de codificación CLI sin interfaz gráfica.

Resumen

LLMOrchestrator es un módulo Go independiente y reutilizable para crear, gestionar y comunicarse con agentes CLI sin interfaz gráfica (OpenCode, Claude Code, Gemini, Junie, Qwen Code) mediante un protocolo híbrido de tuberías y archivos, con interruptores de circuito por agente, selección multi-proveedor conectable, una abstracción de internacionalización desacoplada y garantías de pruebas anti-engaño.

Descripción breve

Un módulo Go reutilizable que ofrece una interfaz unificada para crear y controlar múltiples agentes CLI impulsados por LLM a través de un protocolo híbrido de tuberías y archivos. Agrupación de agentes con seguridad para hilos, interruptores de circuito y estrategias de enrutamiento seleccionables, deliberadamente agnóstico al consumidor, con un traductor de internacionalización conectable.

Descripción detallada

LLMOrchestrator es una infraestructura compartida para orquestar agentes de codificación CLI sin interfaz gráfica: la estructura básica que todo sistema multiagente necesita en silencio y que suele reconstruirse de manera deficiente. En lugar de que cada proyecto vuelva a implementar el lanzamiento de procesos, el enmarcado de mensajes y el análisis de resultados para herramientas como OpenCode, Claude Code, Gemini CLI, Junie y Qwen Code, proporciona una única interfaz Agent, un AgentPool seguro para hilos y un MultiProviderPool que gestiona

agentes de varios proveedores tras una única fachada. El enrutamiento es conectable mediante un AgentSelector: round-robin que omite proveedores que no cumplen requisitos, o con orden de preferencia y respaldo, de modo que la distribución del trabajo es una política que se elige, no un supuesto codificado. Cada agente concreto es un adaptador ligero sobre un BaseAdapter compartido que gestiona todo el ciclo de vida del proceso: inicio con configuración de tuberías, parada con SIGTERM seguido de SIGKILL, reinicio y verificación de actividad —la parte engorrosa y propensa a errores, resuelta de una vez por todas.

La comunicación es deliberadamente híbrida, adaptando el transporte a la tarea. Un transporte por tuberías maneja mensajes JSON delimitados por saltos de línea con un plazo de lectura por solicitud y un límite de longitud de respuesta para mensajes interactivos rápidos, mientras que un transporte por archivos utiliza directorios de entrada/salida/compartidos por sesión para artefactos grandes o duraderos que no deben residir en una tubería. La resiliencia no es un añadido, sino una característica estructural: un interruptor de circuito por agente se abre tras tres fallos consecutivos con un enfriamiento de 60 segundos antes de una prueba en estado semiabierto, y un monitor de salud en segundo plano verifica el estado de los agentes para que uno caído pueda recuperarse sin esperar a que el tráfico entrante lo detecte. La adquisición de agentes en el grupo se bloquea mediante una variable de condición en lugar de consumir CPU en espera activa, y el analizador de respuestas es sin estado y seguro para llamadas concurrentes. El módulo está estrictamente desacoplado —no se permite que detalles específicos del consumidor se filtren— y cada cadena visible para el usuario pasa por un Translator de internacionalización conectable, con un NoopTranslator que devuelve los identificadores de mensaje tal cual para que una traducción faltante se note claramente en lugar de ocultarse.

Por qué lo creamos

Todo sistema multiagente necesita lanzar y comunicarse con agentes CLI de manera fiable. Resolver una y otra vez el lanzamiento, el enmarcado, el análisis y el manejo de fallos en cada proyecto es ineficiente y propenso a errores. LLMOrchestrator centraliza estas funciones en un módulo reutilizable y desacoplado, cuya responsabilidad especializada lo hace reutilizable —y esa reutilización se destruye en el momento en que cualquier detalle específico del consumidor se filtra.

Por qué es un cambio de juego

Convierte “gestionar un ejército de agentes CLI heterogéneos” de un esfuerzo de ingeniería personalizado por proyecto en una simple importación de biblioteca: agrupación, interrupción de circuitos, gestión del ciclo de vida y enrutamiento conectable ya resueltos y probados. Y como sus pruebas anti-engaño ejercitan el sistema real de extremo a extremo en lugar de conformarse con “compila”, obtienes una abstracción en la que puedes confiar para funcionar bajo concurrencia y fallos, no una que solo parezca correcta en un diagrama.

Qué es innovador

- **Protocolo híbrido tubería+archivo:** velocidad interactiva (líneas JSON sobre stdin/stdout, plazos de lectura, límites de respuesta) y intercambio duradero basado en archivos (bandeja de entrada/salida/compartida) para grandes artefactos, de modo que nunca tengas que sacrificar latencia por durabilidad o viceversa.
- **Grupo multiproveedor con selectores conectables:** una única fachada sobre múltiples proveedores CLI, con enrutamiento por turnos o basado en preferencias elegido como política en lugar de estar integrado.
- **Interruptor de circuito por agente + monitor de salud en segundo plano:** degradación y recuperación automáticas (3 fallos → 60 s abierto → sonda semiabierta), de modo que un agente inestable se aísla y luego se reintegra sin intervención manual.
- **Agrupación sin espera activa:** Acquire se bloquea en sync.Cond hasta que un agente compatible y saludable se libera o el contexto se cancela, por lo que esperar no consume CPU.
- **Desacoplamiento estricto + i18n anti-engaño:** el NoopTranslator devuelve los IDs de mensaje tal cual, de modo que una traducción faltante es imposible de pasar por alto en lugar de quedar en blanco silenciosamente.
- **Seguridad por defecto:** una lista blanca de rutas de binarios evita la interpolación de shell y, por tanto, cualquier superficie de inyección de comandos, respaldada por protección contra recorrido de rutas, un límite de 1 MiB en las respuestas para evitar salidas descontroladas y el enmascaramiento de claves API en los registros.
- **Arnés de desafío anti-engaño:** pruebas reales de disco/JSON/analizador en cinco idiomas, con una puerta de mutación emparejada que debe salir con código distinto de cero cuando la función está rota —una prueba que demuestra que realmente puede fallar—.

Mayores desafíos técnicos y cómo los resolvimos

- **E/S confiable de procesos agente.** Comunicarse con un proceso CLI generado es engañosamente difícil; se resolvió con un transporte híbrido tubería+archivo, un contrato de mensaje/analizador definido para que ambas partes coincidan en el formato de transmisión, y un BaseAdapter que centraliza todo el ciclo de vida del proceso, incluyendo un tiempo de espera elegante para SIGTERM que escala a un SIGKILL como respaldo.
- **Concurrencia sin espera activa.** Se resolvió con un AgentPool basado en mutex y variable de condición, donde Acquire permanece en espera hasta que un agente con las capacidades requeridas se libera, junto con un analizador sin estado y libre de efectos secundarios que es seguro invocar desde múltiples gorutinas simultáneamente.
- **Aislamiento de fallos del proveedor.** Se resolvió para que un proveedor defectuoso no arrastre a los demás: los interruptores de circuito por agente contienen el radio de explosión, y una gorutina de monitor de salud impulsa la recuperación incluso cuando no llegan solicitudes que la activen.
- **Demostrar corrección, no solo compilación.** Se resolvió con un ejecutor de desafíos: docenas de invariantes en en/sr/ja/es/de que ejercitan el sistema real, más una puerta de mutación emparejada (LLMORCH_MUTATE_RUNNER=1 debe fallar → salida del wrapper 99) que rompe deliberadamente la función para demostrar que la puerta en sí no es un engaño.
- **Localización sin fallos silenciosos.** Se resolvió con la costura del NoopTranslator de IDs literales y la inyección de traductores por consumidor, de modo que cualquier laguna en las traducciones siempre sea visible en lugar de ocultarse.

Pila tecnológica

- **Go (1.25)** — seleccionado por su excelente concurrencia y control limpio de procesos, exactamente lo que exige la orquestación de procesos de agentes en vivo; implementa el módulo, sus adaptadores de agentes, transportes y analizador.
- **Solo stdlib de Go (+ testify, yaml.v3)** — una elección deliberada para mantener la superficie de dependencias al mínimo y no incorporar *ningún* SDK de LLM, de modo que el módulo siga siendo ligero y embebible en cualquier consumidor sin arrastrar equipaje de proveedores.
- **Transporte por tuberías (JSON-lines sobre stdio)** — elegido para mensajería interactiva rápida, reforzado con plazos de lectura y límites de longitud de respuesta para que

un agente bloqueado o descontrolado no pueda paralizar al llamante.

- **Transporte por archivos (inbox/outbox/compartido)** — seleccionado para el intercambio duradero de artefactos grandes por sesión, donde una tubería sería la herramienta equivocada.
- **sync.Mutex/sync.Cond** — elegidos para implementar la adquisición bloqueante y equitativa de un grupo de agentes sin espera activa.
- **Cortacircuitos + HealthMonitor** — combinados para ofrecer resiliencia por agente y recuperación activa, no solo detección de fallos.
- **Traductor de pkg/i18n** — seleccionado como capa de localización desacoplada que mantiene las cadenas específicas del consumidor fuera del núcleo.
- **Arnés de pruebas (challenges/runner) + Makefile (test -race, fuzz, cover)** — elegidos para una verificación basada en evidencia y sin engaños, que incluye detección de condiciones de carrera y fuzzing del analizador, de modo que la corrección se demuestre bajo condiciones adversas, no se asuma.

Estado y notas de transparencia

- **Estado: beta.** Un módulo reutilizable y desacoplado, consumido como submódulo por múltiples proyectos Helix/vasic. **Licencia: Apache-2.0**; el repositorio GitHub es público.
- Los metadatos del modelo provienen de LLMsVerifier, puenteados a través de HelixQA; este módulo no importa directamente LLMsVerifier/VisionEngine/DocProcessor. Las pilas tecnológicas referenciadas en el CLAUDE.md de la aplicación principal (Gin/PostgreSQL/etc.) describen helix_code, no este módulo.

Nivel de prioridad: Helix-principal (clúster de infraestructura LLM — módulo reutilizable desacoplado). Se sitúa después de HelixTrack.